

Compte Rendu IPO: The Submerged City

I – Partie 1

I.A) Auteur(s)

LAM-DETRAIT Thibault

Groupe 12

I.B) Thème (phrase-thème validée)

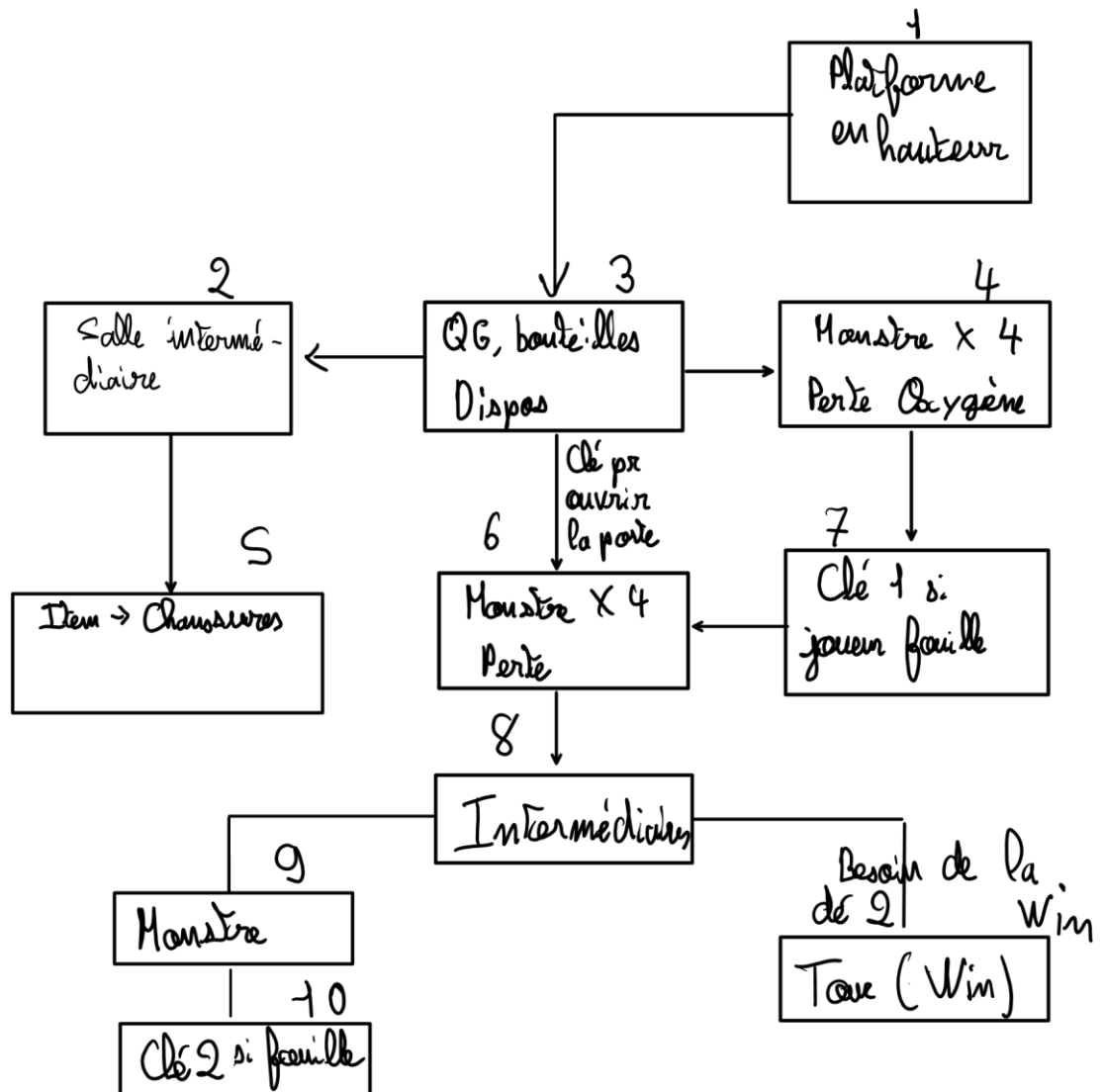
Dans une cité sous-marine abandonnée dans les années 1800, un explorateur doit cartographier cette cité et s'en échapper.

I.C) Résumé du scénario (complet)

(Petit Résumé)

Le joueur arrive sur la première située en hauteur puis saute jusqu'à la salle 3. Le joueur s'il veut peut aller trouver les palmes qui l'aideront dans l'aventure et doit aller récupérer la clé 1 (en passant par des monstres) et ouvrir la porte. Il doit ensuite récupérer la clé 2 en passant par des monstres et enfin accéder à la tour et gagner la partie.

I.D) Plan (complet, avec indication de la partie "réduit" si exercice 7.3.3)



I.E) Scénario détaillé (complet, avec indication de la partie "réduit" si exercice 7.3.3)

J'avais donc pensé pour ce jeu que l'explorateur devait se rendre sur une case précise où se trouverait la tour de la ville sous-marine infestée de monstres. Son but est alors de se rendre sur cette salle avec moult obstacles sur son chemin pour accomplir son objectif avoir une vue d'ensemble sur la ville pour ainsi la cartographier.

Je pensais avoir environ 10 lieux et j'envisageais également trouver un moyen d'implémenter un système d'oxygène qui forcera le joueur à se rendre sur des salles qui lui rempliront son oxygène. S'il il ne le fait pas assez, il perdra la partie car à chaque déplacement, le joueur perdra un petit peu d'oxygène. Le joueur devra donc bien penser à s'approvisionner en oxygène pour continuer l'aventure sachant qu'il ne pourra pas transporter sur lui plus d'une bouteille.

J'ai pensé que sur une salle, il y'aura écrit un panneau sur lequel il est dit que le joueur devra récupérer un objet s'il veut passer à la salle suivante pour ne pas perdre la partie.

Il y'aura quelque salle avec des monstres qui auront pour seul but d'épuiser le joueur car il devra courir pour fuir et donc consommer plus son oxygène.

Pour accéder à la tour le joueur devra trouver une clé qui se situera dans une des salles.

Pour gagner le personnage ne devra pas que se rendre dans la tour mais devra aussi répondre à des questions sur ce qu'il a trouvé dans la cité comme le nombre de monstre et le nombre de salles pour s'assurer que le joueur a bien exploré.

Il y'aura aussi une option fouille dans chaque salle sans monstre qui permettra au joueur de trouver un objet s'il y'en a un. Mais cela lui fera consommer 10 d'oxygène aussi.

I.F) Détail des lieux, items, personnages

Le personnage commencera avec 100 d'oxygène et à chaque déplacement il en perdra 10 s'il croise un monstre il en perdra alors 40. À chaque fois qu'il revient au QG sa bouteille se remplit. Il y'a donc 2 clés qui l'aideront à ouvrir des portes nécessaires à la victoire. Le joueur peut également trouver des chaussures aquatiques qui lui permettront de réduire de moitié l'énergie qui perdra s'il rencontre un monstre.

I.G) Situations gagnantes et perdantes

Situation Gagnante : Le joueur trouve la clé 2 et arrive à la tour avec de l'oxygène et cartographie la cité en répondant aux questions demandées

Situation Perdante : Le joueur se déplace sans oxygène et suffoque, le joueur n'a pas assez d'oxygène en rencontrant un monstre et se fait attraper.

I.H) Eventuellement énigmes, mini-jeux, combats, etc.

Pas d'énigme particulière à part trouver les clés et se souvenir des pièces

I.I) Commentaires (ce qui manque, reste à faire, ...)

Il manque à coder le système d'oxygène

II. Partie 2 : Réponses aux exercices (à partir de l'exercice 7.5 inclus)

7.5 –

L'exercice 7.4 a été complètement terminé. On nous demande de créer la procédure `printLocationInfo()` car à plusieurs reprises dans ce code on répète la même section de code sans la rappeler ce qui faciliterai donc grandement la lecture, la compréhension du code et permet aussi de l'optimiser en évitant la duplication de code. Grâce à cette procédure on a juste besoin d'appeler cette dernière à chaque fois que l'on a besoin de voir quelles sont les différentes sorties.

7.6 –

Pour faire l'exercice 7.6 j'ai donc ajouté l'accessor `getExit()` qui permettait de simplifier le code de la procédure `goRoom()` qui transforme un gros bloc de ifs en une seule ligne de code.

Les attributs sont initialisés à null par défaut donc affecter directement un paramètre null revient au même que de laisser l'attribut comme il est. `this.northExit = north;` fonctionne dans tous les cas car elle définit la pièce si elle existe ou alors met null si elle n'existe pas. Donc tester si chaque paramètre est null n'est pas nécessaire pour le bon fonctionnement de ce code.

Corriger le Unknown, j'ai opté pour rajouter une condition if et else dans la procédure `goRoom()` :

```

if (vNextRoom != null){
    this.aCurrentRoom = vNextRoom;
}
else {
    System.out.println("Unknown direction !");
}

printLocationInfo();

```

7.7-

On va donc réutiliser le code présent dans printLocationInfo(), le couper et crée une nouvelle methode cette fois ci dans Room.

```

public String getExitString()
{
    if(this.getExit("north") != null) {
        System.out.print("north ");
    }
    if(this.getExit("east") != null) {
        System.out.print("east ");
    }
    if(this.getExit("south") != null) {
        System.out.print("south ");
    }
    if(this.getExit("west") != null) {
        System.out.print("west ");
    }
    return null;
}

```

On va alors appeler cette méthode dans la méthode printLocationInfo()

Séparer les classes où les Rooms sont affiché et où les informations sur les Rooms sont crée permet d'éviter le couplage et donc en séparer ces classes permet également à la classe Game de ne pas interférer dans la déclaration des Rooms qui se produit maintenant dans Room.

7.8-

Les Hasmaps nous permettent alors de retirer les restreintes des 4 uniques directions. On peut maintenant créer n'importe quel type de sortie pour une salle car il prend comme paramètre, seulement la direction et la salle que cette direction va amener.

Ex = « bas », v2

La méthode setExits devient alors inutile et peut être remplacée directement par une autre méthode ci-dessous :

```

public Room setExits(String pDirection, Room pClose)
{
    return aExits.put(pDirection, pClose);
}

```

7.9-

On applique alors les modifications de l'exercice 7.9 sur la méthode `getExitString()` en utilisant la classe `Set` importée par la ligne : `import java.util.Set;`

En utilisant la nouvelle forme de `for` découverte, on obtient cette méthode :

```
public String getExitString()
{
    String vReturnString = "Exits: ";
    Set<String> keys = aExits.keySet();

    for(String exit: keys)
    {
        vReturnString += " " + exit;
    }

    return vReturnString;
}
```

7.10-

Cette méthode sert à construire une phrase listant toutes les directions possibles depuis la pièce dans laquelle le joueur se trouve. Elle initialise d'abord une chaîne de caractères avec le mot "Exits:", puis elle récupère toutes les clés de la `HashMap aExits`, donc les directions (« down » , « north » , etc....) et les ajoute une par une à la suite de la phrase avec la nouvelle boucle `for` ce qui donne au final la phrase avec toutes les différentes possibilités.

7.10.2-

En générant la Javadoc, on remarque que la méthode `Game` contient beaucoup moins de méthode que `Room` car `Game` utilise les méthodes déjà déclarée dans `Room`.

7.11-

On ajoute alors la méthode `getLongDescription()` et on modifie dans `Game` `printLocationInfo` de manière à appeler `getLongDescription` dans cette dernière. Ça nous donne alors :

```

/**
 * Affiche une description longue
 */
public String getLongDescription()
{
    return "You are " + aDescription + ".\n" + getExitString();
}

```

```

/**
 * Affiche la description de la pièce courante et ses sorties disponibles.
 */
public void printLocationInfo()
{
    System.out.println(aCurrentRoom.getLongDescription());
}

```

7.14-

Pour créer la nouvelle commande look, on ajoute d'abord dans la String des commandes valides « look », pour qu'elle soit détectée. On crée alors une méthode look qui se contentera de print Location info et enfin on rajoute une condition dans la méthode processCommand de la class Game pour le cas ou cette commande est utilisée. On y écrira :

```

else if (vCommandWord.equals("look")) {
    this.look();
    return false;
}

```

7.15-

On fait alors de même pour la commande eat qui elle se contentera d'afficher dans la console la phrase « You have eaten now and you are not hungry any more. »

7.16-

Pour accomplir cet exercice on va alors créer dans ComandWords une nouvelle méthode showAll qui se fera appelée plus tard par la classe Parser

```

/**
 * Print les différentes commandes valides
 */
public void showAll()
{
    for(String command : aValidCommands)
    {
        System.out.print(command + " ");
    }
    System.out.println();
}

```

showCommands aura alors pour unique but d'afficher les commandes valides automatiquement lorsque l'on rajoute des commandes. Sans écrire une par une les commandes comme avant

```
/**
 * Print une string de commandes valides
 */
public void showCommands()
{
    this.aValidCW.showAll();
}
```

Enfin on va appeler showCommands dans la procédure printHelp en remplaçant le texte montrant les commandes par aParser.showCommands() ;

7.18-

On a donc modifié showAll qui devient getCommandList de manière à séparer les tâches et de faire afficher la liste de commande seulement dans la classe Game. Car auparavant showAll et showCommand avait pour but d'afficher ce qui était demandé et maintenant avec ces méthodes modifiées, elles se contentent seulement de retourner la liste de commandes pour qu'enfin la classe Games les affiche quand cela est nécessaire. De la même manière que l'on a fait getExitString().

7.18.1-

Il y'a alors comme différence entre zuul-better et mon code :

- les commandes look et eat ne sont pas présentes dans le zuul
- la carte est très différente car zuul garde celle de l'université
- le nom des variables et les fonctions ne sont pas écrit exactement de la même manière

7.18.3-

Toutes les images ont été trouvées. Elles sont dans un dossier pour les stocker pour l'instant.

7.18.4-

Le nom que j'ai décidé de donner pour ce jeu est : « The Submerged City »

7.18.6-

Pour incorporer les modifications en étudiant le projet zuul with images on va d'abord ajouter dans la classe Room le nouvel attribut, almageName et on ajoute également la nouvelle méthode getImageName()


```
public String getImageName()
{
    return this.aImageName;
}
```

Avec ces changements, on est donc obligé de modifier la méthode createRooms() dans la classe Game pour ajouter les images. On a donc

```
Room vStart = new Room ("in the entrance of the submerged city", "Panorama.jpg");
Room v2 = new Room ("in an empty street with beautiful buildings", "Road.jpg");
Room v3 = new Room ("in the General Quarter, there is oxygen in this place", "Rest.jpg");
Room v4 = new Room (textMonster, "Monster.jpg");
Room v5 = new Room (textChest, "Chest.jpg");
Room v6 = new Room (textMonster, "Monster.jpg");
Room v7 = new Room (textChest, "Chest.jpg");
Room v8 = new Room ("in an empty street with beautiful buildings", "Road.jpg");
Room v9 = new Room (textMonster, "Monster.jpg");
Room v10 = new Room (textChest, "Chest.jpg");
Room vVictory = new Room ("in the tower you wanted to find for so long", "LightHouse.jpg");
```

Le Scanner devient inutile car maintenant on va utiliser en plus du texte différents éléments comme les images. Puisque que le Scanner ne sert que pour le textuel il semble logique de passer sur les Tokens qui permettront d'utiliser les images.

Par la suite on va créer la nouvelle classe Game Engine qui se composer de la majorité des méthodes de Game. On va aussi incorporer des nouvelles méthodes en utilisant aussi la nouvelle classe utilisateur. On voit alors qu'en testant le jeu que l'interface est changée et les images fonctionnent également ainsi que tout les textes en remplaçant System.out.println par this.Gui.println.

7.18.8-

Pour cet exercice, il fallait ajouter des boutons dans la classe UserInterface afin de pouvoir déclencher des commandes du jeu en cliquant, tout en gardant la ligne de saisie fonctionnelle.

On a donc ajouté huit JButton (Help, Go North, Go South, Go East, Go West, Go Up, Go Down, Quit) regroupés dans un JPanel placé juste au-dessus du champ de saisie. On a donc mis un écouteur sur chaque bouton et dans actionPerformed on regarde d'où vient l'événement avec getSource() pour savoir si c'est un bouton ou le texte qui a été utilisé, si c'est un bouton, la commande correspondante est envoyée directement au moteur de jeu, sinon c'est la saisie clavier qui est traitée normalement. Enfin, les boutons sont aussi désactivés en fin de partie via la méthode enable() qui « éteigne » les boutons quand le jeu est finis.

7.19.2-

Pour réussir cet exercice on va donc mettre toutes les images à la racine du projet dans un fichier nommé Images. Mais le jeu ne va pas trouver ces images si l'on ne change pas la méthode

showImage dans la classe UserInterface. On va donc ajouter au début de la méthode le répertoire dans lequel il se trouve.

```
*/  
public void showImage( final String pImageName )  
{  
    String vImagePath = "Images/" + pImageName;  
    URL vImageURL = this.getClass().getClassLoader().getResource( vImagePath );  
    if ( vImageURL == null )  
        System.out.println( "Image not found : " + vImagePath );  
}
```

7.20-

Pour cet exercice, il fallait permettre à chaque pièce de contenir un objet (item), avec une description et un poids, et d'afficher ses informations quand le joueur entre dans la pièce.

On a d'abord créé une nouvelle classe Item représentant un objet du jeu, avec deux attributs : une description et un poids, avec les getters.

```
public class Item  
{  
    private String aDescription;  
    private double aWeight;  
  
    /**  
     * Crée un item avec une description et un poids.  
     */  
    public Item( final String pDescription, final double pWeight )  
    {  
        this.aDescription = pDescription;  
        this.aWeight      = pWeight;  
    }  
  
    /**  
     * Retourne la description de l'item.  
     */  
    public String getDescription()  
    {  
        return this.aDescription;  
    }  
  
    /**  
     * Retourne le poids de l'item.  
     */  
    public double getWeight()  
    {  
        return this.aWeight;  
    }  
}  
// Item
```

Ensuite, la classe Room a été modifiée pour qu'elle puisse avoir un item : on y a ajouté un attribut aItem, une méthode setItem() pour placer un objet dans une pièce, et une méthode getItemString() qui retourne soit la description et le poids de l'item présent, soit le message "No item here." si la

pièce est vide. La méthode `getLongDescription()` a également été mise à jour pour inclure cette information à chaque fois qu'une pièce est décrite. On n'initialise pas `altm` dans le constructeur donc par défaut une pièce n'a pas d'item ce qui est exactement ce qu'on veut.

```
public void setItem( final Item pItem )
{
    this.aItem = pItem;
}

public String getItemString()
{
    if ( this.aItem == null )
        return "No item here.";
    else
        return "Item: " + this.aItem.getDescription()
            + " (weight: " + this.aItem.getWeight() + "kg)";
}
```

Enfin, dans `GameEngine`, des items ont été créés et placés dans certaines pièces via `setItem()`, laissant les autres volontairement vides.

```
Item vFins    = new Item( "a pair of perfectly fine diving fins, maybe i could help you",    1.0 );
Item vKey1    = new Item( "a mysterious key, it seems like it could open a door",    0.1 );
Item vKey2    = new Item( "a mysterious key, it seems like it could open the lighthouse door", 0.1 );
Item vBottle  = new Item( "a marvelous bottle of oxygen", 2.0 );

v5.setItem( vFins );
v7.setItem( vKey1 );
v10.setItem( vKey2 );
v3.setItem( vBottle );
```

7.20-

La description d'un item devrait être produite par la classe `Item`, avec une méthode comme `toString()`. C'est elle qui possède les données (description, poids), donc c'est elle qui est la mieux placée pour les mettre en forme.

```
public String toString()
{
    return this.aDescription + " (weight: " + this.aWeight + "kg)";
}
```

La classe `Room` va appeler cette méthode dans `getItemString()` pour mettre l'information à la description longue de la pièce. C'est son rôle de savoir ce qu'elle contient.

```
public String getItemString()
{
    if ( this.aItem == null )
        return "No item here.";
    else
        return "Item: " + this.aItem.toString();
}
```

Enfin, c'est GameEngine qui affiche le résultat avec `this.aGui.println()`, car c'est lui qui gère le jeu et les interactions avec Room ducoup on a fait n sorte qu'Item n'affiche rien directement

7.22-

Pour cet exercice, il faut modifier la classe Room pour qu'elle puisse contenir plusieurs items, en utilisant une HashMap. Similaire à ce qu'on avait utilisé pour les Exits

```
private String aDescription;
private HashMap<String, Room> aExits;
private String aImageName;
private HashMap<String, Item> aItems;

public Room (final String pDescription, final String pImage)
{
    this.aDescription = pDescription;
    this.aExits        = new HashMap<String, Room>();
    this.aImageName    = pImage;
    this.aItems        = new HashMap<String, Item>();
}
```

On a ajouté l'attribut `aItems` de type `HashMap<String, Item>` et on l'a mis dans le constructeur. La méthode `addItem()` utilise `put()` et `getItemString()` parcourt les valeurs de la map via `.values()` pour afficher tous les items présents, ou "No item here." si elle est vide avec `.isEmpty()`. Utiliser une HashMap est pratique car on pourra plus tard retrouver un item par son nom, ce qui sera utile quand on ajoutera les commandes `take` et `drop`.

7.22-

Pour respecter les consignes de cet exercice nous allons donc rajouter un objet dans la salle de départ, `vStart`. Puis nous allons ajouter un deuxième objet dans une des salles avec la clé pour respecter la deuxième consigne.

```

Item vFins    = new Item( "Fins", "a pair of perfectly fine diving fins, maybe it could help you move around the city",    1.0 );
Item vKey1    = new Item( "Key1", "a mysterious key, it seems like it could open a door",    0.1);
Item vKey2    = new Item( "Key2", "a mysterious key, it seems like it could open the lighthouse door", 0.1 );
Item vBottle  = new Item( "Bottle", "a marvelous bottle of oxygen", 2.0 );
Item vTelescope = new Item( "Telescope", "a beautiful telescope in which you could see the whole city", 5.0 );
Item vTrash   = new Item( "Trash", "just some useless trash sitting next to the key", 10.0);

vStart.addItem( vTelescope );
v7.addItem( vTrash );
v5.addItem( vFins );
v7.addItem( vKey1 );
v10.addItem( vKey2 );
v3.addItem( vBottle );

```

En testant dans le jeu, on a ceci



```

- a marvelous bottle of oxygen: Bottle (weight: 2.0kg)
> go east
You are in a room with a horrifying monster.
Exits: south west
No item here.
> go south
You are in a very messy room with a key in the center.
Exits: north west
Items:
- a mysterious key, it seems like it could open a door: Key1 (weight: 0.1kg)
- just some useless trash sitting next to the key: Trash (weight: 10.0kg)

```

7.23-

Pour cet exercice, il fallait ajouter une commande back permettant au joueur de retourner dans la pièce précédente.

On a ajouté un attribut aLastRoom dans GameEngine pour mémoriser la pièce dans laquelle le joueur se trouvait avant son dernier déplacement.

```

private Room aCurrentRoom;
private Parser aParser;
private UserInterface aGui;
private Room aLastRoom; ;

```

À chaque fois que le joueur utilise go, la pièce courante est sauvegardée dans aLastRoom avant le changement.

```
private void goRoom(final Command pCommand)
{
    if ( ! pCommand.hasSecondWord() ) {
        this.aGui.println("Go where ?");
        return;
    }

    String vDirection = pCommand.getSecondWord();

    Room vNextRoom = aCurrentRoom.getExit(vDirection);
    if ( vNextRoom == null )
        this.aGui.println( "There is no door!" );
    else {
        this.aLastRoom = this.aCurrentRoom;
        this.aCurrentRoom = vNextRoom;
        this.aGui.println( this.aCurrentRoom.getLongDescription() );
        if ( this.aCurrentRoom.getImageName() != null )
            this.aGui.showImage( this.aCurrentRoom.getImageName() );
    }
}
```

La méthode goBack() permet alors au joueur de retourner dans la pièce précédente. Si aLastRoom est null, cela veut dire que le joueur est encore dans la salle de départ et n'a pas encore bougé. Le jeu va donc lui indiquer.

```
private void goBack()
{
    if ( this.aLastRoom == null ) {
        this.aGui.println( "You can't go back, you are at the beginning!" );
        return;
    }

    Room vTemp = this.aCurrentRoom;
    this.aCurrentRoom = this.aLastRoom;
    this.aLastRoom = vTemp;
    this.aGui.println( this.aCurrentRoom.getLongDescription() );
    if ( this.aCurrentRoom.getImageName() != null )
        this.aGui.showImage( this.aCurrentRoom.getImageName() );
}
```

Il faut aussi ne pas oublier de rajouter le mot back dans les commandes de CommandWords

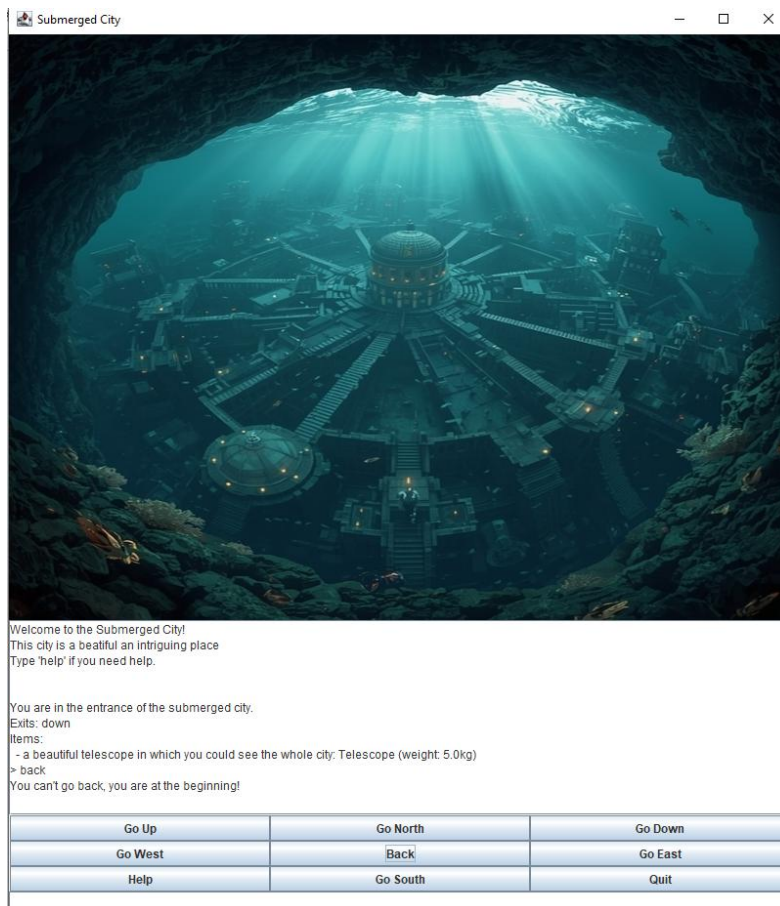
```
private final String[] aValidCommands = { "go", "help", "quit", "look", "eat", "back" };
```

Nous pouvons également ajouté cette commande a un bouton (le bouton look a été remplacé par back).

7.24-

Testons alors la nouvelle commande dans le jeu.

Lorsqu'on utilise back avant de se déplacer, il se passe ceci



Après s'être déplacé, la commande fonctionne bien et nous renvoie dans la salle précédente. En appuyant dessus plusieurs fois d'affilé, on remarque que l'on va alors jongler entre les deux salles.

En testant les commandes, on a remarqué que les commandes fonctionnaient même si l'on rajoutait un second mot. Pour résoudre ce problème, on a modifié la méthode `interpretCommand()` dans `GameEngine` pour créer le cas où il y'a un deuxième mot en utilisant la méthode `hasSecondWord()`, cela nous donne ceci

```
String vCommandWord = vCommand.getCommandWord();
if ( vCommandWord.equals( "help" ) ){
    this.printHelp();
}
else if ( vCommandWord.equals( "go" ) ){
    this.goRoom( vCommand );
}
else if ( vCommandWord.equals( "look" ) ) {
    if ( vCommand.hasSecondWord() )
        this.aGui.println( "Look what? This command has no second word!" );
    else
        this.printLocationInfo();
}
else if ( vCommandWord.equals( "back" ) ){
    if ( vCommand.hasSecondWord() ){
        this.aGui.println( "Back what? This command has no second word!" );
    }
    else {
        this.goBack();
    }
}
else if ( vCommandWord.equals( "eat" ) ) {
    if ( vCommand.hasSecondWord() )
        this.aGui.println( "Eat what? This command has no second word!" );
    else{
        this.aGui.println("You have eaten now and you are not hungry any more.");
    }
}
else if ( vCommandWord.equals( "quit" ) ) {
    this.quit(vCommand);
}
```


7.25-

Après avoir fait le test proposé à cet exercice, on remarque qu'en utilisant plusieurs fois la commande back après plusieurs déplacements, on va juste revenir à la salle précédente et jongler entre ces deux là et nous allons donc pas revenir sur nos pas ce qui été demandé. Nous devons alors régler ce problème à l'exercice suivant

7.26-

Pour cet exercice, il fallait améliorer la commande back pour permettre de remonter tout l'historique des pièces visitées, et non plus seulement la dernière.

On a remplacé l'attribut aLastRoom par une Stack<Room> nommée aPreviousRooms.

```
1 import java.util.Stack;
2
3 public class GameEngine
4 {
5     private Room aCurrentRoom;
6     private Parser aParser;
7     private UserInterface aGui;
8     private Stack<Room> aPreviousRooms;
```

À chaque déplacement avec go, la pièce courante est empilée avec push() avant de changer de pièce.

```
if ( vNextRoom == null )
    this.aGui.println( "There is no door!" );
else {
    this.aPreviousRooms.push( this.aCurrentRoom );
    this.aCurrentRoom = vNextRoom;
    this.aGui.println( this.aCurrentRoom.getLongDescription() );
    if ( this.aCurrentRoom.getImageName() != null )
        this.aGui.showImage( this.aCurrentRoom.getImageName() );
}
```

Quand le joueur utilise back, on dépile la dernière pièce visitée avec pop() pour y retourner. Si la pile est vide, cela veut dire que le joueur est au point de départ et un message le lui dit.

```
private void goBack()
{
    if ( this.aPreviousRooms.isEmpty() ) {
        this.aGui.println( "You can't go back, you are at the beginning!" );
        return;
    }
    this.aCurrentRoom = this.aPreviousRooms.pop();
    this.aGui.println( this.aCurrentRoom.getLongDescription() );
    if ( this.aCurrentRoom.getImageName() != null )
        this.aGui.showImage( this.aCurrentRoom.getImageName() );
}
```

On peut maintenant revenir en arrière autant de fois qu'il y a eu de déplacements, jusqu'au début du jeu.

7.26.1-

Les 2 javadocs ont été générées.

 progdoc		16/04/2026 19:41	File folder
 userdoc		16/04/2026 19:41	File folder

7.27-

Avant d'effectuer le « refactoring » sur le projet, il faut mettre des tests pour voir si le jeu fonctionne toujours même après des changements

Les tests essentiels à effectuer seraient vérifier que tous les déplacements entre les pièces fonctionnent dans chaque direction disponible, que les commandes de base (go, look, help, quit, back, eat) fonctionnent bien, que les messages d'erreur s'affichent correctement quand une direction est invalide ou qu'une commande est mal utilisée (ex look back, pas de second mots), que les items s'affichent bien à l'entrée dans une pièce et que "No item here." apparaît dans les pièces vides, que la commande back remonte bien l'historique jusqu'au point de départ, et enfin que le jeu se termine avec quit.

7.28-

Peut être qu'à la place de tester une par une les commandes du jeu, on pourrait utiliser un fichier qui utilisera des commandes pré écrites. Il faudra ensuite quelque chose pour vérifier le bon fonctionnement des commandes. Je pense qu'il faudra faire ces modifications dans Parser car elle gère les entrées. Il faudra aussi modifier User Interface puisqu'elle gère les interactions avec le joueur

7.28.1-

Pour cet exercice, il fallait créer une commande test permettant d'exécuter automatiquement une suite de commandes lues depuis un fichier txt, pour tester le jeu sans avoir à tout retaper à la main.

On a ajouté "test" dans la liste des commandes valides de CommandWords,

```
private final String[] aValidCommands = { "go", "help", "quit", "look", "eat", "back", "test" };
```

puis créé une méthode test() dans GameEngine qui lit un fichier .txt ligne par ligne et envoie chaque ligne à interpretCommand() comme si le joueur l'avait tapée.

```

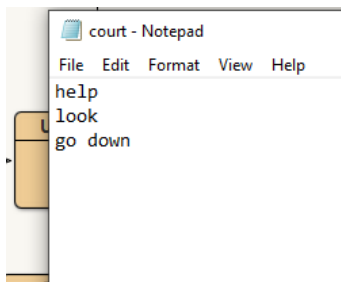
private void test( final Command pCommand )
{
    if ( !pCommand.hasSecondWord() ) {
        this.aGui.println( "Test which file?" );
        return;
    }
    String vFileName = pCommand.getSecondWord();
    try {
        Scanner vScanner = new Scanner( new File( vFileName ) );
        while ( vScanner.hasNextLine() ) {
            String vLine = vScanner.nextLine();
            this.interpretCommand( vLine );
        }
        vScanner.close();
    } catch ( FileNotFoundException e ) {
        this.aGui.println( "File not found: " + vFileName );
    }
}

```

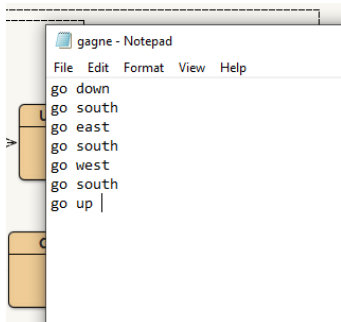
Le nom du fichier est récupéré via le second mot. Il faut donc que l'utilisateur tape le nom du fichier et .txt pour que cela fonctionne. Un try/catch gère le cas où le fichier est introuvable en affichant un message d'erreur sans faire planter le jeu. On a aussi rajouté un scanner pour lire le fichier texte, File pour donner le fichier au scanner et FileNotFoundException pour utiliser catch pour ne pas faire planter le jeu.

7.28.2-

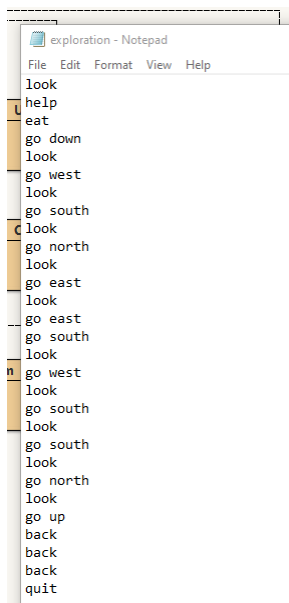
On va maintenant créer 2 autres fichiers pour faire différents tests qui pourront être utiles pour la suite. D'abord on va créer le court qui se contentera que de quelques commandes pour voir si le jeu fonctionne à première vue.



Puis un autre fichier qui sera le chemin vers la victoire nécessaire



et finalement le fichier exploration qui va visiter toutes les salles disponibles du jeu.



```
File Edit Format View Help
look
help
eat
go down
look
go west
look
go south
look
go north
look
go east
look
go east
go south
look
go west
look
go south
look
go south
look
go north
look
go up
back
back
quit
```

7.29-

Pour cet exercice, il fallait effectuer un refactoring en créant une classe Player pour regrouper toutes les informations du joueur, pour que chaque classe ne gère que ce qui la concerne.

On a donc créé la classe Player qui contient le nom du joueur, sa pièce courante aCurrentRoom et la pile aPreviousRooms, qui ont été retirés de GameEngine.

```

9 public class Player
10 {
11     private String aName;
12     private Room aCurrentRoom;
13     private Stack<Room> aPreviousRooms;
14
15     /**
16      * Crée un joueur avec un nom et une pièce de départ.
17      */
18     public Player( final String pName, final Room pStartRoom )
19     {
20         this.aName = pName;
21         this.aCurrentRoom = pStartRoom;
22         this.aPreviousRooms = new Stack<Room>();
23     }
24
25     /**
26      * Retourne la pièce courante du joueur.
27      */
28     public Room getCurrentRoom()
29     {
30         return this.aCurrentRoom;
31     }
32
33     /**
34      * Retourne le nom du joueur.
35      */
36     public String getName()
37     {
38         return this.aName;
39     }
40
41     /**
42      * Déplace le joueur vers la pièce indiquée et empile la pièce courante.
43      */
44     public void goRoom( final Room pNextRoom )
45     {
46         this.aPreviousRooms.push( this.aCurrentRoom );
47         this.aCurrentRoom = pNextRoom;
48     }
49
50     /**
51      * Retourne en arrière dans la pièce précédente.
52      */
53     public Room goBack()
54     {
55         if ( this.aPreviousRooms.isEmpty() ){
56             return null;
57         }
58         this.aCurrentRoom = this.aPreviousRooms.pop();
59         return this.aCurrentRoom;
60     }
61 }

```

Les méthodes `goRoom()` et `goBack()` ont été réparties entre les deux classes. `GameEngine` détermine la pièce suivante et affiche les informations et `Player` effectue le déplacement et gère l'historique. Toutes les références à `aCurrentRoom` dans `GameEngine` ont été remplacées par `aPlayer.getCurrentRoom()`.

On va alors faire beaucoup de changements dans `GameEngine` comme retirer les attributs plus nécessaires et les remplacer par `aPlayer`

```

private Player aPlayer;
private Parser aParser;
private UserInterface aGui;

```

Créer le joueur avec la pièce de départ

```

this.aPlayer = new Player( "Hero", vStart );

```

`Player` fait donc le déplacement et `GameEngine` print l'info

```

this.aPlayer.goRoom( vNextRoom );
this.printLocationInfo();

```

7.30-

Pour cet exercice, il fallait permettre au joueur de ramasser et déposer un seul item à la fois avec les commandes take et drop.

On a ajouté un attribut aCarriedItem dans la classe Player pour stocker l'item actuellement porté par le joueur,

```
private Item aCarriedItem;
```

initialisé à null au départ.

```
public Player( final String pName, final Room pStartRoom )
{
    this.aName      = pName;
    this.aCurrentRoom = pStartRoom;
    this.aPreviousRooms = new Stack<Room>();
    this.aCarriedItem = null;
}
```

La méthode takeItem() vérifie d'abord que le joueur ne porte pas déjà quelque chose, puis retire l'item demandé de la pièce courante

```
public String takeItem( final String pItemName )
{
    if ( this.aCarriedItem != null ){
        return "You are already carrying an item, drop it first!";
    }
    Item vItem = this.aCurrentRoom.removeItem( pItemName );
    if ( vItem == null ){
        return "There is no item called '" + pItemName + "' here!";
    }
    this.aCarriedItem = vItem;
    return "You picked up " + vItem.toString();
}
```

via une nouvelle méthode removeItem() ajoutée dans Room

```
public Item removeItem( final String pItemName )
{
    return this.aItems.remove( pItemName );
}
```

qui utilise HashMap.remove() pour le supprimer.

La méthode dropItem() fait l'inverse en déposant l'item porté dans la pièce courante et en remettant aCarriedItem à null.

```

public String dropItem()
{
    if ( this.aCarriedItem == null ){
        return "You are not carrying anything!";
    }
    this.aCurrentRoom.addItem( this.aCarriedItem );
    String vMessage = "You dropped: " + this.aCarriedItem.toString();
    this.aCarriedItem = null;
    return vMessage;
}

```

Les deux nouvelles commandes ont été ajoutées dans CommandWords

```

private final String[] aValidCommands = { "go", "help", "quit", "look", "eat", "back", "test", "take", "drop" };

```

et gérées dans GameEngine

```

else if ( vCommandWord.equals( "take" ) ) {
    this.takeItem( vCommand );
} else if ( vCommandWord.equals( "drop" ) ) {
    this.dropItem( vCommand );
}

```

via deux nouvelles méthodes takeItem() et dropItem().

```

private void takeItem( final Command pCommand )
{
    if ( !pCommand.hasSecondWord() ) {
        this.aGui.println( "Take what?" );
        return;
    }
    this.aGui.println( this.aPlayer.takeItem( pCommand.getSecondWord() ) );
}

private void dropItem( final Command pCommand )
{
    if ( pCommand.hasSecondWord() ) {
        this.aGui.println( "Drop what? Just type 'drop' with no second word!" );
        return;
    }
    this.aGui.println( this.aPlayer.dropItem() );
}

```

En faisant ces modifications, la commande drop et take ne marchaient toujours pas

```

You are in the General Quarter, there is oxygen in this place.
Exits: east south west
Items:
- a marvelous bottle of oxygen: Bottle (weight: 2.0kg)
> take Bottle
There is no item called 'Bottle' here!

```

Après avoir pensé que le problème était les majuscules, et après avoir mis les méthodes take et drop dans Room en minuscule, cela ne marchait toujours pas.

Finalement, ce fut juste une erreur dans la création d'item. pName et pDescription n'étaient pas à la bonne place ce qui avait pour cause de mettre comme nom la description et inversement.

Finalement, les commandes fonctionnent maintenant

```
> take Telescope  
You picked up Telescope: a beautiful telescope in which you could see the whole city (weight: 5.0kg)
```

7.31-

Pour cet exercice, l'attribut `aCarriedItem` a été remplacé par une `HashMap<String, Item>` nommée `aCarriedItems` dans `Player`

```
import java.util.Stack;  
import java.util.HashMap;  
  
private String aName;  
private Room aCurrentRoom;  
private Stack<Room> aPreviousRooms;  
private HashMap<String, Item> aCarriedItems;  
  
public Player( final String pName, final Room pStartRoom )  
{  
    this.aName = pName;  
    this.aCurrentRoom = pStartRoom;  
    this.aPreviousRooms = new Stack<Room>();  
    this.aCarriedItems = new HashMap<String, Item>();  
}
```

permettant de porter autant d'items que souhaité. La méthode `takeItem()`

```
public String takeItem( final String pItemName )  
{  
    Item vItem = this.aCurrentRoom.removeItem( pItemName );  
    if ( vItem == null ){  
        return "There is no item called '" + pItemName + "' here!";  
    }  
    this.aCarriedItems.put( vItem.getName(), vItem );  
    return "You picked up: " + vItem.toString();  
}
```

ne vérifie

plus s'il y'a plus qu'un item et ajoute l'item à la salle. La méthode `dropItem()`

```
private void dropItem( final Command pCommand )  
{  
    if ( !pCommand.hasSecondWord() ) {  
        this.aGui.println( "Drop what?" );  
        return;  
    }  
    this.aGui.println( this.aPlayer.dropItem( pCommand.getSecondWord() ) );  
}
```

a maintenant besoin d'un second mot pour savoir quel item déposer.

Finalement on teste les modifications en essayant de déposer le premier objet dans la deuxième salle et cela fonctionne.

7.31.1-

Pour cet exercice, il faut créer une nouvelle classe ItemList qui va être utilisé par les classes Room et Player et qui contient des méthodes utiles pour le jeu.

```
public class ItemList
{
    private HashMap<String, Item> aItems;

    /**
     * Crée une liste d'items vide.
     */
    public ItemList()
    {
        this.aItems = new HashMap<String, Item>();
    }

    /**
     * Ajoute un item à la liste.
     */
    public void addItem( final Item pItem )
    {
        this.aItems.put( pItem.getName(), pItem );
    }

    /**
     * Retire et retourne un item par son nom, ou null si introuvable.
     */
    public Item removeItem( final String pItemName )
    {
        return this.aItems.remove( pItemName );
    }

    /**
     * Retourne true si la liste est vide.
     */
    public boolean isEmpty()
    {
        return this.aItems.isEmpty();
    }
}
```

Cette classe utilise une HashMap<String, Item> et utilise les méthodes addItem(), removeItem(), isEmpty() et getItemString().

Room et Player utilisent maintenant cette nouvelle classe ce qui permet d'éviter la duplication de code.

7.32-

Pour cet exercice, il fallait empêcher le joueur de porter des items au-delà d'un poids maximum. On a ajouté deux attributs dans Player : aMaxWeight qui définit le poids maximum autorisé passé en paramètre du constructeur, et aCurrentWeight

```
private double aMaxWeight;
private double aCurrentWeight;
```

qui suit le poids total actuellement porté. Dans takeItem(), on vérifie que le poids de l'item ne ferait pas dépasser la limite


```

if ( this.aCurrentWeight + vItem.getWeight() > this.aMaxWeight ) {
    this.aCurrentRoom.addItem( vItem );
    return "Too heavy! You can only carry " + (this.aMaxWeight - this.aCurrentWeight) + "kg more.";
}

```

si c'est le cas l'item est remis dans la pièce et un message l'indique. aCurrentWeight est mis à jour à chaque take et drop. En rajoutant (this.aMaxWeight - this.aCurrentWeight) lorsque le joueur essaye de porter un objet trop lourd, on peut voir le poids qu'il peut encore porter.

7.33-

Pour cet exercice, il fallait afficher tous les items portés par le joueur ainsi que leur poids total. Donc on a ajouté une méthode getItemString() dans ItemList

```

public String getItemString( final String pEmptyMessage, final String pPrefix )
{
    if ( this.aItems.isEmpty() ){
        return pEmptyMessage;
    }
    double vTotalWeight = 0.0;
    String vResult = pPrefix;
    for ( Item vItem : this.aItems.values() ) {
        vResult += "\n - " + vItem.toString();
        vTotalWeight += vItem.getWeight();
    }
    vResult += "\nTotal weight: " + vTotalWeight + "kg";
    return vResult;
}

```

qui parcourt les objets du joueur, construit la liste des items et fait le calcul du poids total que le joueur est en train de porter. Cette méthode prend deux paramètres : un message à afficher si la liste est vide, et un préfixe. Player utilise cette méthode via getCarriedItemsString(),

```

public String getCarriedItemsString()
{
    return this.aCarriedItems.getItemString(
        "You are not carrying anything.",
        "You are carrying (" + this.aCurrentWeight + "/" + this.aMaxWeight + "kg):"
    );
}

```

et GameEngine appelle cette méthode lors de la commande items.

```

} else if ( vCommandWord.equals( "items" ) ) {
    this.aGui.println( this.aPlayer.getCarriedItemsString() );
}

```

On reteste le jeu et tout fonctionne bien.

7.34-

Pour cet exercice, il fallait étendre la commande eat pour qu'elle accepte un second mot désignant un item porté par le joueur, et créer un item spécial dont l'effet est de doubler le poids maximum transportable.

La commande eat dans GameEngine

```

}else if ( vCommandWord.equals( "eat" ) ) {
    if ( vCommand.hasSecondWord() )
        this.aGui.println( this.aPlayer.eatItem( vCommand.getSecondWord() ) );
    else
        this.aGui.println( "Eat what?" );
}

```

a été modifiée pour transmettre le second mot à une nouvelle méthode eatItem() dans Player.

```

public String eatItem( final String pItemName )
{
    Item vItem = this.aCarriedItems.removeItem( pItemName );
    if ( vItem == null )
        return "You are not carrying '" + pItemName + "'!";

    if ( vItem.getName().equals( "Cookie" ) ) {
        this.aMaxWeight *= 2;
        return "You ate the magic cookie! You can now carry " + this.aMaxWeight + "kg.";
    }

    this.aCurrentWeight -= vItem.getWeight();
    return "You ate " + vItem.getName() + ". Hmm, not sure that was a good idea...";
}

```

Cette méthode retire l'item de la liste des items portés, vérifie si c'est le cookie magique et dans ce cas double aMaxWeight, sinon elle consomme simplement l'item en mettant à jour aCurrentWeight en soustrayant le poids de l'objet qui vient d'être consommé. Le cookie a été ajouté dans createRooms() avec un poids très léger.

```

Item vCookie = new Item( "Cookie", "a magic cookie, it looks delicious", 0.1 );
vStart.addItem( vTelescope );
v3.addItem( vCookie );

```

En testant dans le jeu, cette nouvelle fonctionnalité marche bien. On pourra aussi l'utiliser pour consommer d'autres items si besoin.

7.34.1-

Les fichiers de tests ont été mis à jour pour incorporer toutes les nouvelles fonctionnalités faites entre temps

7.34.2-

Pour cet exercice, des commentaires ont été ajoutés pour chaque méthode et la javadoc a été Régénérée.

7.42-

Pour cet exercice, il faut donc ajouter un moyen de mettre une limite de temps dans le jeu, pour cela on a utilisé le scénario et l'item mis dans la salle de repos, la bouteille d'oxygène pour le déplacement. On a donc rajouté 2 attributs dans la classe Player, aMovesLeft et aHasOxygen qui représentent respectivement le nombre de déplacement restant ainsi que si le joueur a consommé une bouteille d'oxygène.

```
private int aMovesLeft;           // oxygene restant
private boolean aHasOxygen;       // boolean qui vaut true si le joueur a une l
```

Au début du jeu le joueur sera limité à 5 déplacements et lorsque ce dernier trouvera la bouteille et la consommera, il cela lui ajoutera 30 déplacements possible avant de lui faire perdre la partie. Pour cela on va utiliser une nouvelle méthode déclarée toujours dans aPlayer qui décrémente le compteur a chaque déplacement et retourne true si le joueur est à court d'oxygène. Dans GameEngine, après chaque déplacement dans goRoom(), on va vérifier si le joueur est à court d'oxygène : si c'est le cas on affiche un message de game over et on désactive l'interface avec enable(false), sinon on affiche le nombre de mouvements restants. La bouteille d'oxygène était déjà placée dans le QG (v3) depuis un ancien exercice, donc le joueur doit la trouver et l'utiliser rapidement pour avoir suffisamment d'oxygène pour terminer l'aventure.

```
if ( this.aPlayer.decrementMoves() ) {
    this.aGui.println( "You ran out of oxygen... Game over." );
    this.aGui.enable( false );           //"eteint le jeu"
} else {
    this.aGui.println( "Oxygen left: " + this.aPlayer.getMovesLeft() + " moves." );
}
```

7.42.2-

Pour l'instant, nous nous contenterons de l'IHM actuelle.

7.43-

Pour cet exercice, il fallait ajouter des portes à sens unique dans le jeu, et de faire en sorte que la commande back ne permette pas de revenir en arrière si c'est une trap door.

Dans createRooms(), on met la sortie uniquement depuis la pièce de départ sans mettre la sortie inverse, ce qui permet de créer une trap door (On l'avait déjà fait inconsciemment, il n'y avait aucun de remonter dans la pièce départ). Ajouter une nouvelle trap door ne nécessite donc qu'une seule ligne de code, ce qui respecte bien la contrainte demandée.

Pour que back ne puisse pas franchir une trap door à l'envers, on a ajouté une méthode isExit() dans Room qui vérifie si une pièce est bien accessible depuis la pièce courante. Dans goBack() de Player, on regarde d'abord si on peut revenir en arrière avant de dépiler la pièce précédente. Si ce n'est pas possible à cause d'une trap door, un message spécifique s'affiche pour prévenir le joueur.

```
private void goBack()
{
    Room vPrevious = this.aPlayer.goBack();
    if ( vPrevious == null ) {
        if ( this.aPlayer.isPreviousRoomsEmpty() ) {
            this.aGui.println( "You can't go back, you are at the beginning!" );
        } else {
            this.aGui.println( "You can't go back, there is a trap door!" );
        }
        return;
    }
    this.printLocationInfo();
}
```

Deux trap doors ont été ajoutées dans createRooms() avec chacune une seule ligne de code ce qui a été demandé.

7.43.1-

On a rajouté des commentaires pertinents sur les méthodes et généré les 2 javadocs

7.44-

Pour cet exercice, il fallait ajouter un téléporteur qui peut être ramassé et puis être chargé (charge) dans une pièce puis déclenché (fire) depuis une autre pour s'y téléporter.

On a créé une classe Beamer qui hérite de Item, ce qui permet de le ramasser et de le porter comme n'importe quel objet du jeu. Elle gère elle-même la pièce mémorisée via charge() qui sauvegarde la pièce courante, et fire() qui retourne la pièce mémorisée et remet le téléporteur à zéro comme ça le joueur peut le recharger dans la pièce de son choix .

Les méthodes chargeBeamer() et fireBeamer() ont été ajoutées dans Player car c'est lui qui porte et utilise les items. Dans GameEngine, deux nouvelles méthodes gèrent les commandes charge et fire en appelant les méthodes de Player et en affichant le résultat. Le déplacement lors du fire est aussi sauvegardé dans la pile de Player pour que la commande back fonctionne toujours après s'être téléporté. Ce choix d'implémentation permet d'avoir plusieurs téléporteurs sans modifier le reste du code. Ça fonctionnerait également avec plusieurs joueurs puisque chaque Beamer gère sa propre pièce mémorisée.

On a aussi rajouté dans CommandWords les nouvelles commandes et aussi des nouvelles conditions dans interpretCommands ce qui nous permet d'utiliser ces nouvelles fonctionnalités du Beamer.

7.45 (optionnel)

Pour cet exercice, il fallait ajouter des portes verrouillées qui demandaient une clé spécifique pour l'ouvrir.

On a créé une classe Door qui relie deux pièces et stocke le nom de la clé nécessaire. Dans Room, on a ajouté une HashMap<String, Door> où la clé est le nom de la clé requise, ce qui permet de retrouver directement une porte par le nom de sa clé. Chaque porte est ajoutée dans les deux pièces qu'elle relie grâce à addDoor(), ce qui permet de la verrouiller des deux côtés. Dans goRoom(), on vérifie d'abord si la direction existe puis si une porte verrouillée bloque le passage. La commande unlock permet d'ouvrir une porte avec la bonne clé, qui est conservée après usage mais qu'on peut tout de même jeter. Les deux portes du jeu sont la porte entre v3 et v6 ouverte avec Key1, et la porte entre v8 et vVictory ouverte avec Key2. On a aussi ajouté la nouvelle commande unlock dans interpretCommand et CommandWords. Puisqu'il y'a une trap door entre v4 et v7, j'ai trouvé ça pratique de mettre une door en v6 ce qui nous permet de revenir en v3 ce qui n'était pas possible avant due à la trap door.

7.45.1-

On a mis à jour les fichiers de test qui cette fois ci test toutes les nouvelles possibilités ajouté grâce à ces derniers exercices

7.45.2-

On a rajouté des commentaires pertinents sur les méthodes et généré les 2 javadocs

7.46-

Pour cet exercice, il fallait créer une pièce spéciale qui téléporte le joueur aléatoirement quand il en sort.

Après avoir lu le chapitre 9.11, on a découvert que la meilleure façon de faire ça est d'utiliser l'héritage. On a donc créé une classe `TransporterRoom` qui hérite de `Room`. La méthode `getExit()` est redéfinie avec `@override` pour retourner une pièce aléatoire au lieu de la pièce dans la direction demandée.

Grâce au polymorphisme, `GameEngine` n'a pas besoin d'être modifié du tout. Quand `goRoom()` appelle `getExit()`, Java va appeler la version redéfinie de `TransporterRoom` plutôt que celle de `Room`.

On a aussi créé une classe `RoomRandomizer` séparée pour gérer la logique aléatoire avec `Random` et `nextInt()`.

On a aussi ajouté une nouvelle image pour la nouvelle salle.

7.46.1-

Pour cet exercice il fallait créer une commande `alea` qui permet de contrôler où le joueur sera téléporté quand il sort de la `TransporterRoom`, mais seulement quand on est en train d'exécuter un fichier de test.

Pour faire ça on a ajouté un attribut `aForcedRoom` dans `RoomRandomizer`. Quand cet attribut n'est pas null, `getRandomRoom()` retourne cette pièce au lieu d'en choisir une au hasard. Sinon il choisit normalement une pièce aléatoire.

Dans `GameEngine` on a ajouté un attribut `aTestMode` qui devient `true` quand on lance un fichier de test avec la commande `test`, et redevient `false` à la fin. La commande `alea` vérifie d'abord si on est en mode test, et si ce n'est pas le cas elle affiche un message d'erreur. Si on est bien en mode test, `alea v3` va forcer la prochaine téléportation vers `v3`, et `alea` tout seul remet le tirage en mode vraiment aléatoire.

C'est pratique parce que maintenant dans les fichiers de test on peut prévoir exactement où le joueur va être après la `TransporterRoom`.

7.46.3-

Les commentaires javadocs ont bien été faits

7.46.4-

Les javadocs ont bien été générés

Bonus

Vu que pour l'instant le jeu n'avait pas de fin. Un if qui vérifie si la salle dans laquelle se trouve le joueur est la salle de victoire. Vu que nous n'avions ni utilisé les monstres ni les bottes, on va ajouter une fonctionnalité pour les deux. Comme prévu dans le scénario, avoir un monstre dans sa salle va augmenter le nombre d'oxygène dépensé par le joueur. On a ajouté un attribut `aHasMonster` dans `Room` avec les méthodes `setMonster()` et `hasMonster()`, et on a marqué les pièces `v4`, `v6` et `v9` comme contenant un monstre dans `createRooms()`. Dans `Player`, on a ajouté un attribut `aHasFins` qui devient `true` quand le joueur ramasse les palmes dans `takeItem()` et repasse à `false` s'il les dépose dans `dropItem()`. La méthode `monsterPenalty()` retire 2 déplacements d'oxygène sans palmes et seulement 1 avec, et retourne un message différent dans chaque cas. Dans `goRoom()` de `GameEngine`, après chaque déplacement on vérifie si la nouvelle pièce contient un monstre et on affiche le message retourné par `monsterPenalty()`.

IV – Déclaration anti-plagiat

Je n'ai recopié aucune ligne de code à part si c'était demandé dans l'exercice ni utilisé chatGPT pour la réalisation et la conception du code de ce devoir ainsi que pour le scénario. Cependant j'ai utilisé de l'IA générative pour les images du jeu.